

## SIMULACRO EXAMEN PRÁCTICO: Sistema Básico de Control de Vuelos y Operaciones para un Aeropuerto

|     |                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.  | <b>Título del caso:</b>                                                      | Sistema Básico de Control de Vuelos y Operaciones para un Aeropuerto                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 2.  | <b>N2 Áreas de la carrera:</b>                                               | <ol style="list-style-type: none"><li>1. Tecnologías de la Información</li><li>2. Diseño y Desarrollo de Software</li></ol>                                                                                                                                                                                                                                                                                                                                                                                                    |
| 3.  | <b>N3 Subáreas de la carrera relacionadas con el perfil de egreso:</b>       | <ol style="list-style-type: none"><li>1. Análisis de Datos y Sistemas</li><li>2. Base de Datos</li><li>3. Programación</li><li>4. Calidad y Gestión</li></ol>                                                                                                                                                                                                                                                                                                                                                                  |
| 8.  | <b>N4 Materias:</b>                                                          | <ol style="list-style-type: none"><li>1. Base de Datos I</li><li>2. Base de Datos II</li><li>3. Programación III</li><li>4. Programación IV</li><li>5. Sistemas Operativos</li></ol>                                                                                                                                                                                                                                                                                                                                           |
| 9.  | <b>Identificación de resultados de aprendizaje relacionados con el caso:</b> | <p>El estudiante demuestra que es capaz de:</p> <ul style="list-style-type: none"><li>• Diseñar y administrar bases de datos SQL y NoSQL.</li><li>• Implementar un backend con API REST.</li><li>• Consumir servicios desde aplicaciones web y móviles.</li><li>• Aplicar principios básicos de seguridad y validación.</li><li>• Ejecutar pruebas unitarias.</li><li>• Configurar y ejecutar un sistema en Linux.</li><li>• Integrar todas las capas del sistema de forma funcional.</li></ul>                                |
| 10. | <b>Objetivo</b>                                                              | Desarrollar, documentar y presentar una solución de software completa que permita gestionar servicios internos de una organización, integrando todas las capas del stack tecnológico (front-end, back-end, bases de datos SQL y NoSQL, seguridad, pruebas y despliegue en Linux), mediante metodologías de gestión de proyectos y herramientas colaborativas.                                                                                                                                                                  |
| 11. | <b>Planteamiento:</b>                                                        | <p>Diseñe e implemente un Sistema Básico de Control de Vuelos y Operaciones para un Aeropuerto, integrando una base de datos relacional, una base de datos no relacional, un backend con API, una interfaz web, una aplicación móvil y su ejecución en un entorno Linux. El sistema debe permitir registrar vuelos y su asignación a puertas de embarque, consultar su estado y almacenar eventos operativos, demostrando la integración completa del stack tecnológico y la aplicación de buenas prácticas de desarrollo.</p> |

|     |                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12. | Descripción de entregables | <p><b>Back-end funcional</b></p> <ul style="list-style-type: none"> <li>• API REST u otra arquitectura definida</li> <li>• Validaciones, control de acceso, sanitización</li> </ul> <p><b>Front-end funcional</b></p> <ul style="list-style-type: none"> <li>• Interfaz implementada</li> <li>• Integración con API</li> <li>• Manejo de estados y errores</li> </ul> <p><b>Proyecto Móvil funcional</b></p> <ul style="list-style-type: none"> <li>• Interfaz implementada</li> <li>• Integración con API</li> <li>• Manejo de estados y errores</li> </ul> <p><b>Base de datos relacional (SQL)</b></p> <ul style="list-style-type: none"> <li>• Scripts de creación</li> <li>• Consultas necesarias</li> </ul> <p><b>Base de datos NoSQL</b></p> <ul style="list-style-type: none"> <li>• Colecciones, documentos</li> <li>• Consultas</li> </ul> <p><b>Integración completa del sistema</b></p> <ul style="list-style-type: none"> <li>• Front-end ↔ API</li> <li>• Proyecto Móvil ↔ API</li> <li>• API ↔ SQL y NoSQL</li> <li>• Flujo end-to-end funcionando</li> </ul> <p><b>Ejecución en Linux</b></p> <ul style="list-style-type: none"> <li>• Comandos utilizados</li> <li>• Configuración necesaria</li> <li>• Evidencia de funcionamiento en terminal</li> </ul> |
|-----|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## CASO AEROPUERTO: Sistema Básico de Control de Vuelos y Operaciones para un Aeropuerto

**Regla de negocio:** gestionar puertas de embarque y vuelos con estados (SCHEDULED, BOARDING, DEPARTED, DELAYED, CANCELLED) y registrar operación (aerolíneas y eventos) en NoSQL.

### MODELO DE DATOS (2 tablas + 2 colecciones)

Tablas PostgreSQL (2):

#### Tabla gates (puertas de embarque):

```
id BIGSERIAL PK
code VARCHAR(10) NOT NULL UNIQUE
terminal VARCHAR(20) NOT NULL
is_available BOOLEAN NOT NULL DEFAULT TRUE
created_at TIMESTAMP NOT NULL DEFAULT NOW()
```

#### Tabla flights (vuelos):

```
id BIGSERIAL PK
gate_id BIGINT NOT NULL REFERENCES gates(id)
flight_number VARCHAR(20) NOT NULL (ej: "AA1234")
```

```
destination VARCHAR(100) NOT NULL
status VARCHAR(20) NOT NULL (SCHEDULED|BOARDING|DEPARTED|DELAYED|CANCELLED)
departure_time TIMESTAMP NOT NULL
created_at TIMESTAMP NOT NULL DEFAULT NOW()
```

Colecciones MongoDB (2):

#### **Colección airlines (aerolíneas registradas):**

```
_id ObjectId
name string
code string (ej: "AA")
country string
is_active bool
created_at date
```

#### **Colección flight\_events (eventos operativos):**

```
_id ObjectId
flight_id long (id SQL)
event_type string (CREATED|BOARDING_STARTED|DEPARTED|DELAYED|CANCELLED)
source string (WEB|MOBILE|SYSTEM)
note string (texto simple, sin objeto)
created_at date
```

**Nota de integración:** flight\_events.flight\_id debe guardar el id del vuelo creado en PostgreSQL para vincular operación con el vuelo.

## **SECUENCIA LÓGICA**

1. Crear BD y usuario en PostgreSQL
2. Crear proyecto Django y modelos
3. Migrar (Django crea tablas)
4. Verificar tablas en psql
5. Crear endpoints SQL (gates, flights)
6. Crear DB y usuario en Mongo
7. Crear colecciones e índices
8. Crear endpoints Mongo (airlines, flight-events)
9. Integrar: al crear flight en SQL generar event en Mongo
10. ReactJS consume SQL (gates y flights)
11. Mobile consume Mongo (airlines y flight-events)
12. Evidencia en Linux + Git

## **MATERIAS Y PREGUNTAS**

### **BASE DE DATOS I – PostgreSQL (8 preguntas)**

1. Crear la base de datos airport\_db en PostgreSQL desde terminal/psql.
2. Crear el usuario backend\_user con contraseña segura y sin superusuario.
3. Asignar permisos mínimos para migraciones Django (connect, schema usage, create/alter).
4. Verificar desde psql conexión con backend\_user y existencia de airport\_db.
5. Ejecutar migraciones Django y verificar que existen las tablas gates y flights (listar tablas y un select).
6. Crear índice en flights(status) y demostrar uso con EXPLAIN o consultas (select \* from flights;).
7. Crear vista vw\_active\_flights que liste vuelos en estado SCHEDULED/BOARDING.

8. Crear una función o trigger (uno): Función: total de vuelos por estado, o Trigger: impedir departure\_time anterior a created\_at en DB.

## **BASE DE DATOS II – MongoDB (7 preguntas)**

1. Crear/definir DB airport\_logs (mostrar use airport\_logs en mongosh).
2. Crear usuario mongo\_backend\_user con contraseña exa\_2026\_ute.
3. Asignar roles mínimos para leer/escribir en airport\_logs (sin admin global).
4. Probar autenticación con mongo\_backend\_user y verificar operación en airport\_logs.
5. Verificar/crear colecciones airlines y flight\_events con los campos definidos.
6. Crear índice en flight\_events(flight\_id) y evidenciar con getIndexes().
7. Ejecutar 2 consultas: eventos por flight\_id; eventos por rango de fechas (created\_at).

## **PROGRAMACIÓN III – Backend Django + Frontend React (9 preguntas)**

### **Backend – Django REST**

1. Crear proyecto Django airport\_backend.
2. Crear app airport.
3. Configurar conexión a PostgreSQL (airport\_db).
4. Configurar conexión a MongoDB (airport\_logs).
5. Crear modelos Django para gates y flights (según campos).
6. Crear endpoint GET /gates (lista puertas de embarque) desde SQL.
7. Crear endpoint GET y POST /flights (listar y crear vuelo) desde SQL, y en POST generar un evento en flight\_events (Mongo) con flight\_id.

### **Frontend – ReactJS (consume TABLAS SQL)**

1. Crear proyecto React.
2. Crear vista que consuma GET /gates y GET /flights y muestre puertas y vuelos, con estados y manejo de carga/error.

## **PROGRAMACIÓN IV – React Native (9 preguntas)**

Objetivo: Consumir la API desde una app móvil básica enfocada a NoSQL.

1. Crear proyecto React Native.
2. Configurar conexión con la API del backend.
3. Crear pantalla principal.
4. Consumir endpoint GET /airlines (Mongo) y mostrar aerolíneas.
5. Consumir endpoint GET /flight-events (Mongo) y mostrar eventos (por flight\_id o últimos).
6. Mostrar aerolíneas y eventos en listas (puede ser navegación simple).
7. Manejar estado de carga.
8. Manejar errores de conexión.
9. Evidenciar ejecución en emulador o dispositivo.

## **SISTEMAS OPERATIVOS – Ubuntu 24.04 en VM (8)**

### **1. Crear estructura del proyecto**

Crear una carpeta llamada examen, dentro de ella una carpeta aeropuerto, y dentro de aeropuerto las carpetas: backend, frontend, movil, docs. Luego verificar la estructura.

Comandos a usar:

```
cd, mkdir -p, tree
```

### **2. Navegación y verificación**

Entrar a la carpeta aeropuerto, verificar ubicación actual y listar el contenido.

Comandos a usar:

`cd, pwd, ls -la`

### 3. Crear archivos de evidencia

Dentro de docs crear los archivos comandos.txt y evidencia.txt, luego registrar la fecha dentro de evidencia.txt y verificar contenido.

Comandos a usar:

`touch, date >>, cat`

### 4. Redirección de salida

Guardar la salida de who en un archivo y luego agregar la salida de ls -la al mismo archivo. Verificar el contenido final.

Comandos a usar:

`who, ls -la, >, >>, cat`

### 5. Buscar palabra dentro de archivo

Inicio archivo

```
Proyecto Aeropuerto - Backend
GET /api/flights/
GET /api/flights/15/
POST /api/flights/
DELETE /api/flights/10/
INFO: flight created successfully
INFO: flights service running
WARN: flight delay detected
```

Final Archivo

Escribir contenido anterior dentro de comandos.txt y luego buscar esa palabra dentro del archivo.

Comandos a usar:

`nano o cat << EOF, grep, grep -n, cat`

### 6. Localizar archivo

Crear un archivo README.md dentro de backend y luego encontrarlo desde la carpeta aeropuerto.

Comandos a usar:

`touch, find, locate (opcional)`

### 7. Copiar y mover archivos

Copiar el archivo evidencia.txt para crear un respaldo y luego mover ese respaldo a otra carpeta del proyecto.

Comandos a usar:

`cp, mv, ls -la`

### 8. Permisos y Sticky Bit

Ver permisos actuales de docs, modificar permisos, crear carpeta shared, aplicar 1777 y verificar el Sticky Bit.

Comandos a usar:

`ls -ld, chmod, mkdir`